

As **coleções** (ou estruturas de dados embutidas) do Python são variáveis capazes de armazenar múltiplos valores dentro de uma única estrutura.

No Python, temos **4 coleções principais: Tuplas, Listas, Dicionários e Conjuntos (Sets)**. Embora todas guardem dados, cada uma funciona de forma diferente e serve para resolver tipos de problemas específicos.

1. Tuplas (`tuple`) — A Coleção Imutável e Protegida

A **tupla** é uma coleção **ordenada** e **imutável**. Definimos uma tupla utilizando **parênteses** `()`.

O que significa ser "Imutável"?

Significa que, depois que você cria uma tupla com determinantes valores, você **nunca mais** poderá adicionar, remover ou alterar o valor de qualquer posição dela enquanto o programa rodar.

Para que serve?

É ideal para dados fixos que não podem sofrer alterações acidentais ao longo do código (ex: coordenadas GPS, dias da semana, dados de configuração).

Operações e Comandos Explicados:

Criação:

Python

```
tupla = ('Homo sapiens', 'Canis familiaris', 'Felis catus')
```

A variável `tupla` armazena 3 textos (strings) em posições fixas.

Acesso por Índice:

Python

```
print(tupla[0]) # Retorna 'Homo sapiens'
```

Cada item dentro de uma tupla ganha um número de posição (índice), começando sempre em `0`:

Posição `0`: 'Homo sapiens'

Posição `1`: 'Canis familiaris'

Posição `2`: 'Felis catus'

Busca de Posição com `.index()`:

Python

```
tupla.index('Canis familiaris') # Retorna 1
```

Você passa o valor exato, e o método `.index()` varre a tupla e devolve em qual índice aquele item está guardado.

Percorrendo com `for`:

Python

```
for elemento in tupla:  
    print(elemento)
```

O laço `for` faz uma repetição automática: cria a variável temporária `elemento`, copia para ela o primeiro item da tupla, roda o `print()`, e depois passa automaticamente para o segundo e terceiro itens até acabar a tupla.

2. Listas (`list`) — A Coleção Flexível e Dinâmica

A **lista** é uma coleção **ordenada** e **mutável**. Definimos uma lista usando **colchetes** `[]`.

O que significa ser "Mutável"?

Ao contrário da tupla, a lista é totalmente dinâmica: você pode adicionar novos itens, remover itens existentes, alterar a ordem e modificar valores quantas vezes quiser.

Para que serve?

É a estrutura mais utilizada no dia a dia quando temos um volume de dados que cresce ou diminui durante a execução do sistema (ex: carrinho de compras, lista de usuários logados).

Operações e Comandos Explicados:

Criação e Concatenação (`+`):

Python

```
l1 = ['Homo sapiens', 'Canis familiaris', 'Felis catus']  
l2 = ['Xenopus laevis', 'Ailuropoda melanoleuca']  
l3 = l1 + l2
```

O operador `+` entre duas listas une (concatena) os itens das duas, gerando uma nova lista chamada `l3` com os 5 elementos em sequência.

Multiplicação/Replicação (`*`):

Python

```
l2_2 = l2 * 2
```

O operador `*` duplica a lista `l2`. Se a lista tinha 2 itens, agora terá 4 itens (os 2 originais repetidos em sequência).

Fatiamento (Slicing [início:fim]):

Python

```
print(l1[0:2]) # Retorna ['Homo sapiens', 'Canis familiaris']
```

Permite extrair um "pedaço" da lista.

Regra Importante do Python: *O índice final não é incluído. Portanto, `0:2` pega os elementos do índice 0 e 1 (para antes de chegar no*

2).

Adicionando itens com `.append()` :

Python

```
l1.append('Gorila gorila')
```

O método `.append()` adiciona o novo item **sempre no final** da lista.

Removendo itens com `.remove()` :

Python

```
l1.remove('Felis catus')
```

O método `.remove()` busca a **primeira ocorrência** desse texto específico dentro da lista e o apaga. Os itens restantes "encostam" para preencher o espaço vago.

Apagando variáveis com `del()` :

Python

```
del(l1)
```

O comando `del` apaga a variável inteira da memória do computador. Se você tentar acessar `l1` após esse comando, o Python emitirá um erro `NameError` .

3. Dicionários (`dict`) — A Mapeadora de Chave e Valor

O **dicionário** é uma estrutura **mutável** baseada no conceito de **Pares de Chave-Valor** (Key-Value). Definimos um dicionário usando **chaves** `{}` e o caractere **dois-pontos** `:` .

Como funciona?

Imagine um dicionário de papel real: você procura uma **palavra** (Chave) para encontrar o seu **significado** (Valor). No Python é igual: em vez de acessar elementos por números de índice (`0` , `1` , `2`), você acessa o dado por uma chave personalizada (geralmente um texto).

Operações e Comandos Explicados:

Criação:

Python

```
coleta = {'Aedes aegypt': 32, 'Aedes albopictus': 22, 'Anopheles darlingi': 14}
```

A chave `'Aedes aegypt'` guarda o número `32` , e assim por diante.

Acessando Valores por Chave:

Python

```
print(coleta['Aedes aegypt']) # Retorna 32
```

Passamos a chave entre colchetes e o Python nos entrega diretamente o valor associado a ela.

Inserção e Modificação:

Python

```
coleta['Rhodnius montenegrensis'] = 11
```

Se a chave 'Rhodnius montenegrensis' **não existir**, o Python a cria com o valor 11. Se ela **já existir**, o Python sobrescreve o valor antigo com o novo.

Remoção com del :

Python

```
del coleta['Aedes albopictus']
```

Remove a chave 'Aedes albopictus' e também o seu valor da memória.

Métodos de Inspeção:

`coleta.keys()` : Retorna uma lista com todas as **chaves** (nomes dos insetos).

`coleta.values()` : Retorna uma lista com todos os **valores** (quantidades numéricas).

`coleta.items()` : Retorna pares no formato de tuplas (chave, valor) .

Fundindo Dicionários com .update() :

Python

```
coleta2 = {'Anopheles gambiae': 13, 'Anopheles deaneorum': 14}
coleta.update(coleta2)
```

O `.update()` copia todos os pares do dicionário `coleta2` para dentro de `coleta`. Se houver chaves repetidas, os valores de `coleta2` prevalecerão.

Iteração com Desempacotamento:

Python

```
for especie, num_especimes in coleta.items():
    print(f'Espécie: {especie}, número de espécimes coletados: {num_especimes}')
```

O método `.items()` permite capturar no `for` **duas variáveis ao mesmo tempo**: `especie` recebe a chave e `num_especimes` recebe o valor em cada volta do laço.

4. Conjuntos (`set`) — A Coleção de Elementos Únicos

O **conjunto** (ou `set`) é uma coleção **não ordenada** que tem como regra absoluta **NÃO permitir elementos duplicados**. Criamos usando a função `set()` ou chaves `{}` contendo apenas elementos separados por vírgulas.

Para que serve?

É perfeito para duas tarefas:

Eliminar repetições de dados de forma automática.

Fazer operações matemáticas de teoria dos conjuntos (intersecção, união, diferença).

Operações e Comandos Explicados:

Eliminação de Duplicatas com `set()` :

Python

```
biomeleculas = ('proteína', 'ácidos nucleicos', 'carboidrato', 'lipídeo',  
               'ácidos nucleicos', 'carboidrato', 'carboidrato', 'carboidrato')  
  
print(set(biomeleculas))
```

Mesmo que 'carboidrato' apareça 4 vezes na tupla original, ao converter para `set()`, o Python descarta todas as repetições e mantém apenas 1 exemplar de cada item: `{'proteína', 'ácidos nucleicos', 'carboidrato', 'lipídeo'}`.

Intersecção (`.intersection()`):

Python

```
c1 = {1, 2, 3, 4, 5}  
c2 = {3, 4, 5, 6, 7}  
c3 = c1.intersection(c2)  
print(c3) # Retorna {3, 4, 5}
```

Mapeia e retorna apenas os elementos que **existem em ambos os conjuntos simultaneamente**.

Diferença (`.difference()`):

Python

```
c1.difference(c2) # Retorna {1, 2}
```

Descobre quais elementos estão no primeiro conjunto (`c1`), mas **NÃO estão** no segundo conjunto (`c2`).

Tabela Comparativa Definitiva

Estrutura	Sintaxe de Criação	Aceita Duplicados?	Pode ser Alterada? (Mutável)	Como Acessar um Item?
Tupla	<code>(1, 2, 3)</code>	Sim	Não	Pelo Índice Numérico <code>tupla[0]</code>)
Lista	<code>[1, 2, 3]</code>	Sim	Sim	Pelo Índice Numérico <code>lista[0]</code>)
Dicionário	<code>{'chave': 'valor'}</code>	Chaves: Não		